

Linear Classifiers: Class Imbalance and Distribution Shifts

Nikos Evangelou

Department of Computer Science, Aristotle University of Thessaloniki, Greece

nevangelou@csd.auth.gr

1. Introduction

This study investigates the mathematical foundations, optimization dynamics, and real-world vulnerabilities of linear classifiers using the MNIST database. We implement Softmax Regression, One-vs-All (OvA), and One-vs-One (OvO) Perceptrons entirely from scratch to validate their underlying mechanics, before transitioning to production-grade frameworks for large-scale evaluation.

The experiments are structured around two primary scenarios:

- **Scenario A (Balanced Baseline):** Evaluates the models under ideal, class-uniform conditions, isolating the optimal feature space representation.
- **Scenario B (Class Imbalance & Data Scarcity):** Simulates a pathological environment characterized by extreme class skewness (20 : 1 ratio) and minority data starvation (50 samples per class).

2. Exploratory Data Analysis

To understand the handwriting patterns and statistical properties of the MNIST dataset, we performed an extensive Exploratory Data Analysis.

2.1. Data Visualization

Figure 1 illustrates random samples from each class. We observe that handwriting styles exhibit significant natural variability.



Figure 1. Visual samples of handwritten digits from the dataset.

2.2. Dimensionality, Pixel Sparsity & Scale

Each image is represented as a 784-dimensional feature vector.

However, the feature space is highly sparse. The histogram in Figure 2 reveals that the vast majority of pixels exhibit 0 intensity level (background), with a secondary peak near 255 (strokes). Since most peripheral pixels carry

redundant or zero variance, we can compress the feature space effectively. That will be done via Principal Component Analysis (PCA).

Furthermore, pixel intensities range continuously from 0 to 255. Normalization (scaling to [0, 1]) is required to ensure numerical stability and fast convergence for gradient-based linear models.

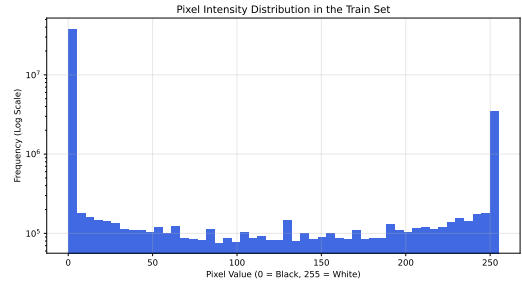


Figure 2. Histogram of pixel intensities across the training set (Log scale).

2.3. Average Class Representation

By calculating the mean of all samples per class, we generate "prototypical" digit templates (Figure 3). While '0' and '7' maintain relatively sharp average forms, digits such as '4', '8', and '9' appear heavily blurred. This blurring confirms high intra-class variance and indicates spatial overlap between classes, suggesting that linear decision boundaries may struggle to separate these overlapping regions.

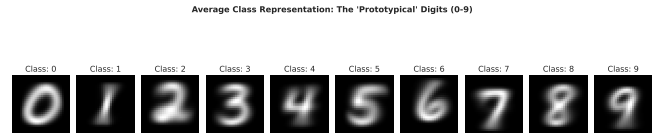


Figure 3. Average digit templates (Centroids) for each class, revealing structural ambiguities.

2.4. Class Distribution (Baseline)

The initial training set (60,000 samples) exhibits a balanced distribution across all 10 digit classes (approximately 6,000 samples per class), as shown in Figure 4. This uniform state serves as the baseline (Scenario A) prior to the artificial introduction of class imbalance (Scenario B).

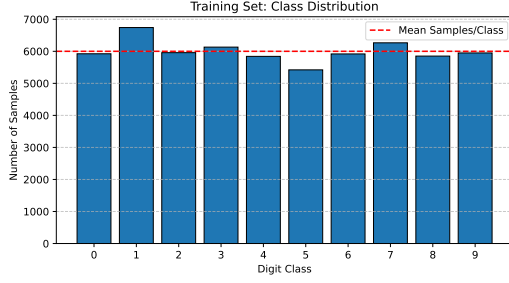


Figure 4. Distribution of digit classes across the baseline MNIST training dataset, demonstrating a balanced pre-experimental state.

3. Data Preprocessing & Subsampling

3.1. Type Casting

The dataset was downcast to float32. While this optimization significantly reduces the memory footprint of the high-dimensional feature matrices, it inherently lowers the numerical overflow threshold.

3.2. Data Normalization

Pixel intensities were normalized to the $[0, 1]$ range by dividing all values by 255.0. This global scaling method is preferred for image data over feature-wise scaling (e.g., MinMaxScaler) because it preserves the relative intensity relationships between pixels and maintains the physical meaning of the image. Furthermore, it avoids computational issues with zero-variance pixels located at the image borders, which are common in the MNIST dataset.

3.3. Subsampling for Scenario A

To establish a controlled, balanced baseline for Scenario A, a stratified random subset was extracted strictly from the training distribution. Exactly 1,000 samples were randomly selected without replacement from each of the 10 digit classes, yielding a perfectly balanced training set of 10,000 samples.

4. Feature Engineering (Theoretical Overview)

4.1. Spatial Transformation (Image Centering)

As shown in the first row of figure 5 many digits are not perfectly aligned within the 28×28 grid, suffering from significant positional offsets. To mitigate this translation variance, we implemented an **Image Centering** stage.

The algorithm calculates the **center of mass** ($\bar{x}, \bar{y}$) for each digit based on pixel intensity distribution. Each sample is then translated so that its center of mass aligns with the geometric center of the canvas (13.5, 13.5). Figure 5 illustrates the visual impact of the centering transformation.

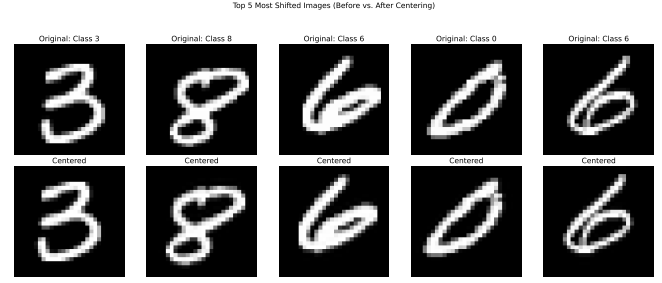


Figure 5. Visual comparison between raw MNIST digits (top) and the resulting images after the centering transformation (bottom).

4.2. Dimensionality Reduction (PCA)

4.2.1. Definition and Algorithmic Context. To address the high dimensionality of the MNIST dataset (784 features), we employ Principal Component Analysis (PCA) as an unsupervised feature extraction technique. The optimal projection subspace will be determined statistically to maximize information retention while filtering background noise.

4.2.2. Variance Analysis and Thresholding. Exploratory analysis of the cumulative explained variance reveals significant structural redundancy in the pixel space. As illustrated in Figure 6, capturing 95% of the total statistical variance requires approximately 152 principal components.

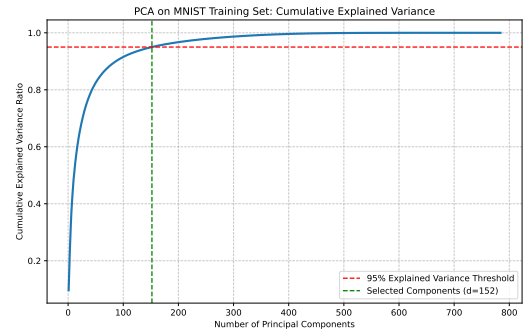


Figure 6. Cumulative explained variance as a function of the number of principal components. The chosen 95% threshold efficiently reduces dimensionality while preserving structural integrity.

4.2.3. Dynamic Implementation Strategy. To ensure a fair experimental baseline and prevent data leakage, the dimensionality reduction transformation is not computed globally. Instead, it is integrated dynamically within all cross-validation folds and hold-out evaluation pipelines. To achieve this, PCA is consistently parameterized by variance ratio ($n_components=0.95$) rather than a hardcoded integer. This forces the algorithm to independently compute the exact number of required components based solely on the specific training subset of each execution fold, strictly protecting the validation sets from structural exposure.

5. Custom Implementations: Linear Classifiers

The implemented algorithms are:

- 1) **Multinomial Logistic Regression (Softmax):** A multiclass linear model utilizing the categorical cross-entropy loss function and gradient descent optimization.
- 2) **Perceptron (One-vs-All / OvA):** An ensemble of $K = 10$ independent binary Perceptrons, where each model separates one class from all the rest. Prediction is determined by the maximum decision boundary margin.
- 3) **Perceptron (One-vs-One / OvO):** An ensemble of $\frac{K(K-1)}{2} = 45$ binary Perceptrons, where a dedicated model is trained for every possible pair of classes. Prediction is determined via a majority voting scheme.

K denotes the number of classes.

5.1. Experimental Setup

To verify the mathematical correctness of our custom implementations, we used a simple stratified 80/20 train-test split. This isolated baseline quickly validates whether the custom gradient updates and loss functions converge properly. As a strict prerequisite, features were scaled but not PCA-reduced, as this step will be validated later in the study.

5.2. Custom Algorithm Implementation Details

- **Binary Perceptron (Figure 7):** The foundation of the OvA and OvO strategies. It utilizes Stochastic Gradient Descent (SGD), shuffling data every epoch, and updates weights sample-by-sample based on the standard Perceptron learning rule. Training halts immediately if perfect linear separation is achieved (zero training error).
- **OvA & OvO Wrappers (Figure 8 & 9):** These function as ensemble wrappers. The OvA architecture instantiates 10 independent binary classifiers (one per digit) and predicts using the maximum decision boundary margin. The OvO architecture instantiates 45 binary classifiers (covering every unique digit pair) and determines the final prediction via a majority voting scheme.
- **Multinomial Logistic Regression (Softmax) (Figure 10):** Implemented utilizing full Batch Gradient Descent. It optimizes the Categorical Cross-Entropy loss function and incorporates an L2 regularization penalty (λ_{reg}) to constrain weight magnitudes.
- **Loss Tracking & Early Stopping:** All algorithms dynamically monitor the training loss. Furthermore, the Softmax implementation exhibits an early stopping mechanism: if the loss on a provided validation

set continuously increases for a predefined number of epochs (patience), training stops to prevent overfitting.

```
class BinaryPerceptron:
    def __init__(self, learning_rate=0.01, max_epochs=1000):
        self.lr = learning_rate
        self.max_epochs = max_epochs
        self.weights = None
        self.bias = 0
        self.n_iter_ = 0
        self.loss_history = []
        self.val_loss_history = []

    def fit(self, X, y, X_val=None, y_val=None):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0
        self.n_iter_ = 0
        self.loss_history = []
        self.val_loss_history = []

        for epoch in range(self.max_epochs):
            self.n_iter_ += 1
            errors = 0

            # Shuffle training data for Stochastic Gradient Descent
            indices = np.arange(n_samples)
            np.random.shuffle(indices)
            X_shuffled = X[indices]
            y_shuffled = y[indices]

            # Training Update (Sample by Sample)
            for idx, x_i in enumerate(X_shuffled):
                linear_output = np.dot(x_i, self.weights) + self.bias
                if y_shuffled[idx] * linear_output <= 0:
                    update = self.lr * y_shuffled[idx]
                    self.weights += update * x_i
                    self.bias += update
                errors += 1

            # Loss Calculation (Perceptron Criterion)
            # Computed on the static (unshuffled) X for consistent loss tracking
            train_dist = y * (np.dot(X, self.weights) + self.bias)
            train_loss = np.mean(np.maximum(0, -train_dist))
            self.loss_history.append(train_loss)

            # Validation Loss Calculation
            if X_val is not None and y_val is not None:
                val_dist = y_val * (np.dot(X_val, self.weights) + self.bias)
                val_loss = np.mean(np.maximum(0, -val_dist))
                self.val_loss_history.append(val_loss)

            if errors == 0:
                break # Perfect linear separation achieved

    def decision_function(self, X):
        return np.dot(X, self.weights) + self.bias

    def predict(self, X):
        return np.where(self.decision_function(X) >= 0, 1, -1)
```

Figure 7. Custom Binary Perceptron utilizing Stochastic Gradient Descent.

5.3. Execution & Visualization

In order to validate the underlying algorithmic mechanics, custom implementations of Softmax, Perceptron One-vs-All (OvA), and Perceptron One-vs-One (OvO) were evaluated on the scaled, full-dimensional dataset (strictly prior to any PCA dimensionality reduction or spatial centering). The results, summarized in **Table 1**, highlight the fundamental trade-offs between computational complexity, memory footprint, and convergence speed.

5.3.1. Performance and Resource Trade-offs Analysis.

- **Time Complexity:** The OvO architecture significantly outperformed OvA in training time (10.48s vs. 383.88s). While OvA forces 10 models to train sequentially on the entire dataset, OvO distributes the workload across 45 models, each training only on a microscopic subset (two classes), leading to rapid convergence.

TABLE 1. PERFORMANCE AND RESOURCE UTILIZATION METRICS FOR THE CUSTOM LINEAR CLASSIFIER IMPLEMENTATIONS.

Model	Accuracy (%)	Convergence (Steps)	Model Memory (KB)	Training Time (sec)
Softmax	90.35	548	61.33	37.60
Perceptron OvA	85.85	6460	61.33	365.60
Perceptron OvO	91.40	988	275.98	9.19

```

class CustomPerceptronOvA:
    def __init__(self, learning_rate=0.01, max_epochs=1000):
        self.lr = learning_rate
        self.max_epochs = max_epochs
        self.models = {}
        self.classes = None
        self.total_steps_ = 0
        self.max_steps_ = 0

    def fit(self, X, y, X_val=None, y_val=None):
        self.classes_ = np.unique(y)
        steps_per_model = []

        # Train a binary classifier for each class against the rest
        for c in self.classes_:
            y_binary = np.where(y == c, 1, -1)

            y_val_binary = None
            if y_val is not None:
                y_val_binary = np.where(y_val == c, 1, -1)

            model = BinaryPerceptron(learning_rate=self.lr, max_epochs=self.max_epochs)
            model.fit(X, y_binary, X_val, y_val_binary)

            self.models[c] = model
            steps_per_model.append(model.n_iter_)

        self.total_steps_ = sum(steps_per_model)
        self.max_steps_ = max(steps_per_model)

    def predict(self, X):
        scores = np.zeros((X.shape[0], len(self.classes_)))
        # Collect decision function margins from all models
        for idx, c in enumerate(self.classes_):
            scores[:, idx] = self.models[c].decision_function(X)
        # Select the class with the highest margin
        return self.classes_[np.argmax(scores, axis=1)]

```

Figure 8. Implementation logic of the One-vs-All (OvA) ensemble wrapper.

- **Memory Footprint:** The speed of OvO comes at a memory cost. Storing 45 distinct binary classifiers requires roughly 4.5 times more memory (275.98 KB) compared to the single weight matrix of Softmax or the 10 models of OvA (61.33 KB).
- **Accuracy:** OvO achieved the highest accuracy (91.00%), as dividing the multi-class problem into smaller, binary sub-tasks simplifies the decision boundaries, making them easier for a linear classifier to approximate in high-dimensional space.

5.3.2. Optimization Dynamics: Softmax Convergence.

The Softmax Loss Curve (Figure 11) provides critical insights into the optimization process:

- **Initialization:** The training loss strictly originates at approximately 2.302. This mathematically validates the initialization state. With 10 mutually exclusive classes, an untrained model assigns a uniform probability of $p = \frac{1}{10} = 0.1$ to each class. The Cross-Entropy loss is thus $-\ln(0.1) \approx 2.302$.
- **Early Stopping & Overfitting Prevention:** The implemented strict early stopping mechanism successfully triggered at epoch 547. While the Training Loss (blue line) continued its asymptotic descent, the Validation Loss (orange line) stabilized around epoch 400 and began to marginally increase, indicating the onset of overfitting. Halting the training

```

class CustomPerceptronOvO:
    def __init__(self, learning_rate=0.01, max_epochs=1000):
        self.lr = learning_rate
        self.max_epochs = max_epochs
        self.models = {}
        self.classes = None
        self.total_steps_ = 0
        self.max_steps_ = 0

    def fit(self, X, y, X_val=None, y_val=None):
        self.classes_ = np.unique(y)
        n_classes = len(self.classes_)
        steps_per_model = []

        # Train a binary classifier for every unique pair of classes
        for i in range(n_classes):
            for j in range(i + 1, n_classes):
                c1, c2 = self.classes_[i], self.classes_[j]

                # Filter Training Set for the two specific classes
                mask = (y == c1) | (y == c2)
                X_pair = X[mask]
                y_pair_raw = y[mask]
                y_binary = np.where(y_pair_raw == c1, 1, -1)

                # Filter Validation Set for the two specific classes
                X_val_pair = None
                y_val_binary = None
                if X_val is not None and y_val is not None:
                    mask_val = (y_val == c1) | (y_val == c2)
                    X_val_pair = X_val[mask_val]
                    y_val_pair_raw = y_val[mask_val]
                    y_val_binary = np.where(y_val_pair_raw == c1, 1, -1)

                model = BinaryPerceptron(learning_rate=self.lr, max_epochs=self.max_epochs)
                model.fit(X_pair, y_binary, X_val_pair, y_val_binary)

                self.models[(c1, c2)] = model
                steps_per_model.append(model.n_iter_)

        self.total_steps_ = sum(steps_per_model)
        self.max_steps_ = max(steps_per_model)

    def predict(self, X):
        n_samples = X.shape[0]
        n_classes = len(self.classes_)
        votes = np.zeros((n_samples, n_classes))
        class_to_idx = {c: i for i, c in enumerate(self.classes_)}

        # Tally votes from all binary classifiers
        for (c1, c2), model in self.models.items():
            predictions = model.predict(X)
            idx_c1 = class_to_idx[c1]
            idx_c2 = class_to_idx[c2]
            votes[predictions == 1, idx_c1] += 1
            votes[predictions == -1, idx_c2] += 1

        # Predict the class with the majority vote
        return self.classes_[np.argmax(votes, axis=1)]

```

Figure 9. Implementation logic of the One-vs-One (OvO) ensemble wrapper.

prevented the model from memorizing the training noise.

5.3.3. Task Difficulty: OvO Separability Analysis. The OvO Task Difficulty chart (Figure 12) effectively illustrates the varying geometric complexity of digit classification:

- **Easy Separability (0 vs 1):** The green curve demonstrates that classifying '0' against '1' is a trivial task for the Perceptron. The model achieved perfect linear separation (zero error) in just under 5 epochs due to their highly distinct spatial footprints.
- **Hard Separability (4 vs 9):** Conversely, the red curve (4 vs 9) exhibits severe oscillation and required approximately 50 epochs to converge. The structural overlap between a '4' and a '9' (often differing only by a top loop closure) creates a highly complex decision boundary, forcing the Perceptron to aggressively and repeatedly adjust its weights.

```

class SoftmaxClassifier:
    def __init__(self, learning_rate=0.1, max_epochs=1000, tol=1e-6, lmbda_reg=0.01, patience=10):
        self.lr = learning_rate
        self.max_epochs = max_epochs
        self.tol = tol
        self.lmbda_reg = lmbda_reg
        self.patience = patience
        self.consecutive_epochs_without_improvement = 0
        self.weights = None
        self.bias = None
        self.x_train = None
        self.y_train = None
        self.x_val = None
        self.y_val = None
        self.loss_history = []
        self.val_loss_history = []
        self.classnames = None

    def softmax(self, x):
        # Numerical stability: subtract max before exponentiating
        exp_z = np.exp(x - np.max(x, axis=1, keepdims=True))
        return exp_z / np.sum(exp_z, axis=1, keepdims=True)

    def fit(self, x, y, x_val=None, y_val=None):
        n_samples, n_features = x.shape
        self.classes = np.unique(y)
        n_classes = len(self.classes)

        self.weights = np.zeros((n_features, n_classes))
        self.bias = np.zeros(n_classes)
        self.x_train = x
        self.y_train = y
        self.x_val = x_val
        self.y_val = y_val

        # Early stopping tracking variables
        val_improvement = 0
        best_val_loss = float('inf')

        # One-hot encode training labels
        y_one_hot = np.zeros((n_samples, n_classes))
        for i, c in enumerate(self.classes):
            y_one_hot[:, i] = (y == c).astype(int)

        # One-hot encode validation labels
        if x_val is not None and y_val is not None:
            x_val_one_hot = x_val.shape[0]
            y_val_one_hot = np.zeros((x_val_one_hot, n_classes))
            for i, c in enumerate(self.classes):
                y_val_one_hot[:, i] = (y_val == c).astype(int)

        for epoch in range(self.max_epochs):
            self.x_iter += 1

            # --- Training Forward Pass ---
            linear_model = np.dot(x, self.weights) + self.bias
            y_predicted = self.softmax(linear_model)

            # Cross-Entropy Loss with L2 Regularization
            data_loss = -np.mean(np.sum(y_one_hot * np.log(y_predicted + 1e-10), axis=1))
            reg_loss = (self.lmbda_reg / 2) * np.sum(self.weights ** 2)
            total_loss = data_loss + reg_loss
            self.loss_history.append(total_loss)

            # --- Validation Forward Pass & Strict Early Stopping ---
            if x_val is not None and y_val is not None:
                val_linear_model = np.dot(x_val, self.weights) + self.bias
                val_y_predicted = self.softmax(val_linear_model)
                val_data_loss = -np.mean(np.sum(y_val_one_hot * np.log(val_y_predicted + 1e-10), axis=1))
                val_reg_loss = (self.lmbda_reg / 2) * np.sum(self.weights ** 2)
                val_total_loss = val_data_loss + val_reg_loss
                self.val_loss_history.append(val_total_loss)

            # Check if validation loss has improved
            if val_total_loss < best_val_loss:
                val_improvement = 1
                best_val_loss = val_total_loss
                self.consecutive_epochs_without_improvement = 0
            else:
                self.consecutive_epochs_without_improvement += 1

            # Terminate if patience is exhausted
            if self.consecutive_epochs_without_improvement >= self.patience:
                print(f"Early stopping triggered at epoch {epoch} to prevent overfitting.")
                break

            # Fall back to tolerance-based stopping once if no validation set is provided
            if epoch >= 500 and len(self.loss_history) > 20 and self.loss_history[-1] < self.tol:
                break

            # --- Backward Pass (Gradient Computation) ---
            dw = (1 / n_samples) * np.dot(x.T, (y_predicted - y_one_hot)) + self.lmbda_reg * self.weights
            db = (1 / n_samples) * np.sum(y_predicted - y_one_hot, axis=0)

            # Weight updates
            self.weights -= self.lr * dw
            self.bias -= self.lr * db

        def predict(self, x):
            linear_model = np.dot(x, self.weights) + self.bias
            probabilities = self.softmax(linear_model)
            return self.classes[np.argmax(probabilities, axis=1)]

```

Figure 10. Implementation of the Multinomial Logistic Regression (Softmax) utilizing Batch Gradient Descent.

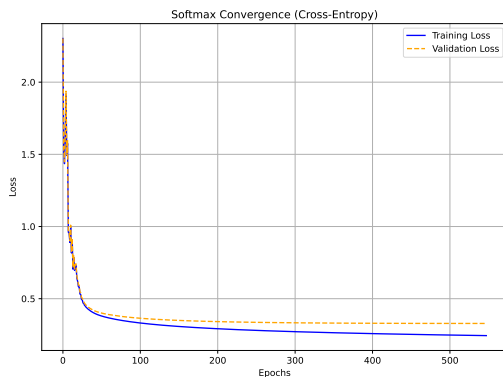


Figure 11. Optimization dynamics: Training versus validation loss convergence for the custom Softmax implementation.

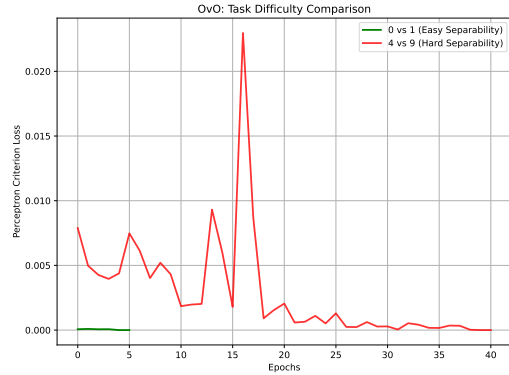


Figure 12. Convergence rate comparison across different digit pairs in the custom OvO Perceptron, highlighting varying levels of linear separability.

6. Transition to Scikit-Learn

Having validated the algorithmic mechanics through custom implementations, all subsequent large-scale experiments utilize the `scikit-learn` framework. This transition is dictated by two key factors:

- **Computational Efficiency:** Exhaustive hyperparameter tuning via Stratified 5-Fold Cross-Validation requires training hundreds of model iterations. Performing this search in pure Python—especially for the 45-model OvO ensemble—is computationally prohibitive compared to `scikit-learn`'s optimized implementation.
- **Optimization Consistency:** Standardized library estimators guarantee strict convergence criteria and numerical stability, ensuring that performance differences stem purely from the tested hyperparameters rather than custom loop overheads.

Consequently, `SGDClassifier(loss='log_loss')` and `Perceptron` from the `sklearn.linear_model` module serve as our standardized estimators for the remainder of this study.

Algorithmic Justification for `SGDClassifier`: Although `LogisticRegression` is the conventional estimator in `scikit-learn`, its default solvers automatically adjust the step size, preventing us from testing specific learning rates. To evaluate how different learning rates (η_0) and training epochs (`max_iter`) affect convergence, we use `SGDClassifier(loss='log_loss')` instead. This configuration implements the exact same Softmax Regression using standard Stochastic Gradient Descent, giving us direct control over the optimization parameters.

7. Experimental Setup

7.1. Feature Space Optimization

To isolate algorithmic performance and eliminate confounding variables, we first determine the optimal data rep-

representation before conducting hyperparameter tuning. Using a balanced 10,000-sample subset and Stratified 5-Fold Cross-Validation, we evaluate the baseline models across four configurations: Raw Pixels, PCA (retaining 95% variance), Centered Pixels, and Centered + PCA.

As detailed in Table 2, *Centered_PCA* offers the best operational balance. While spatial centering alone yields a minor accuracy gain ($\approx 0.5\%$), combining centering with PCA reduces weight matrix memory by over 80% (e.g., OvO memory drops from 138 KB to 21 KB) and accelerates training by $3\times$ to $6\times$. Consequently, *Centered_PCA* is locked as the input representation for all subsequent experiments. Because feature dimensionality remains static after this step, model memory is fixed and therefore omitted from subsequent tuning tables.

7.2. Validation Strategy

To ensure methodological rigor and prevent data leakage, evaluation follows a standardized protocol:

- **Hyperparameter Tuning:** Conducted via Stratified 5-Fold Cross-Validation (*GridSearchCV*) on the training subset to optimize learning rates (η_0), iteration limits (*max_iter*), and regularization strength (α).
- **Evaluation Metrics:** Accuracy is used for the balanced Scenario A, while the Macro-averaged F1-Score is required for the skewed Scenario B to penalize minority-class misclassifications.
- **Data Normalization:** Input features are normalized globally by dividing by 255.0. Because this scaling involves no sample-derived parameters, it introduces zero data leakage.
- **Hold-Out Test Set:** A balanced, untouched test set of 10,000 samples serves as the final real-world benchmark for both scenarios.

8. Scenario A: Standard Classification

8.1. Hyperparameter Tuning

We conduct an exhaustive Grid Search with Stratified 5-Fold Cross-Validation across three architectures: *SGDClassifier* (Softmax), One-vs-All (OvA) Perceptron, and One-vs-One (OvO) Perceptron. The search space evaluates the initial learning rate (η_0), maximum epochs (*max_iter*), and L_2 regularization strength (α), along with constant versus adaptive learning rate schedules for Softmax.

Search Space Constraints: Two specific configurations were deliberately omitted to maintain methodological rigor:

- **Optimal Schedule without Regularization:** In *SGDClassifier*, the 'optimal' schedule uses α to control learning rate decay ($\eta \propto 1/\alpha$). Combining it with *penalty=None* forces α to act solely as a decay parameter rather than a weight penalty, introducing a confounding variable.

- **L_1 Regularization:** Because PCA already eliminates redundant and zero-variance dimensions, applying L_1 feature selection on top of an already compressed orthogonal space is redundant.

8.2. Grid Search Results & Analysis

The hyperparameter tuning results (Table 3) highlight four key insights:

- **OvO Superiority:** The OvO Perceptron achieved the highest cross-validation accuracy (91.53%), outperforming Softmax (90.56%) and OvA (87.59%) by partitioning the multi-class problem into simpler binary boundaries.
- **Implicit Regularization:** All models converged to minimal L_2 penalties ($\alpha \in [10^{-6}, 10^{-5}]$). This confirms that PCA successfully filtered feature noise.
- **Learning Rate Schedule:** For Softmax, a steady step size (constant, $\eta_0 = 0.01$) navigated the orthogonal PCA landscape more reliably than the decaying optimal schedule.
- **Convergence Bottlenecks:** Models converged within 13–15 epochs on average under early stopping (*tol*= $1e-3$). However, complex digit pairs in OvO (such as 4 vs 9) required up to 31 epochs, directly increasing the total training time.

8.3. Model Evaluation & Error Diagnostics

8.3.1. Optimization Dynamics & Step Size. Evaluating gradient trajectories on Softmax (Figure 13) demonstrates the impact of learning rate selection. A constant $\eta_0 = 0.01$ yielded stable convergence, whereas an excessively large rate ($\eta_0 = 1.0$) caused loss oscillations and an undersized rate ($\eta_0 = 10^{-6}$) stalled progress. The optimal schedule decayed too aggressively, plateauing prematurely. (Note: the grid-optimal setup with $\alpha = 10^{-5}$ caused gradient explosion under *float32* precision during epoch 1; $\alpha = 10^{-3}$ was substituted strictly to illustrate schedule dynamics).

8.3.2. Training Epochs and Learning Curves. The epoch-wise accuracy of the OvA Perceptron (Figure 14) displays severe oscillations due to error-driven weight updates, validating the necessity of early stopping via *tol*. Furthermore, the cross-validation learning curves (Figure 15) display a narrow train-validation gap alongside plateauing accuracy. This diagnoses low variance but high bias, confirming that linear hyperplanes have hit their fundamental capacity ceiling on this feature space.

8.3.3. Test Set Performance & Error Diagnostics. Generalization on the hold-out test set matched cross-validation estimates: Perceptron OvO reached 92.00%, Softmax achieved 91.35%, and Perceptron OvA obtained 87.64%.

Confusion matrices (Figure 16) show consistent bottlenecks between morphologically similar classes (e.g., 4

TABLE 2. CROSS-VALIDATION PERFORMANCE AND COMPUTATIONAL REQUIREMENTS ACROSS DISTINCT FEATURE REPRESENTATIONS.

Feature Space	Model	CV Accuracy (%)	Train Time (s)	Memory (KB)
Centered	Perceptron OvA	87.57	2.46	30.70
Raw	Perceptron OvA	87.05	3.17	30.70
Centered PCA	Perceptron OvA	86.96	0.40	4.77
PCA	Perceptron OvA	86.02	0.49	6.02
Centered	Perceptron OvO	91.52	1.86	138.16
Centered PCA	Perceptron OvO	91.24	0.92	21.45
Raw	Perceptron OvO	91.05	1.73	138.16
PCA	Perceptron OvO	90.53	0.92	27.07
Centered PCA	Softmax (SGD)	90.11	1.66	4.73
Centered	Softmax (SGD)	89.91	7.06	30.66
PCA	Softmax (SGD)	89.13	2.19	5.98
Raw	Softmax (SGD)	88.19	7.80	30.66

TABLE 3. OPTIMAL HYPERPARAMETERS AND 5-FOLD CROSS-VALIDATION ACCURACY FOR THE LINEAR CLASSIFIERS EVALUATED IN SCENARIO A.

Model	CV Acc (%)	Best Params	Avg Epochs	Max Epochs	Train Time (s)
Perceptron OvO	91.53	{'alpha': 1e-06, 'eta0': 0.1, 'max_iter': 50, 'penalty': 'l2'}	15.0	31	122.46
Softmax (SGD)	90.56	{'alpha': 1e-05, 'eta0': 0.01, 'learning_rate': 'constant', 'max_iter': 50, 'penalty': 'l2'}	14.0	14	278.39
Perceptron OvA	87.59	{'alpha': 1e-06, 'eta0': 0.1, 'max_iter': 50, 'penalty': 'l2'}	8.8	15	99.24

vs 9 and 3 vs 8). Qualitative inspection of confident misclassifications (Figures 17, 18, and 19) reveals that models fail confidently on legible digits whenever stroke positions align with opposing class weights. Because linear classifiers operate on static pixel overlaps, they cannot model spatial transformations such as rotation or stroke thickness.

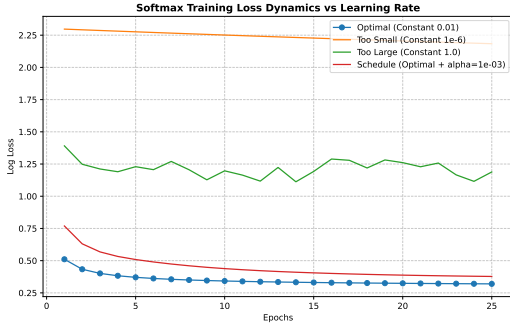


Figure 13. Impact of various learning rates of the SGDClassifier (Log-Loss).

9. Scenario B: Class Imbalance & Data Scarcity

9.1. Problem Setup

To evaluate linear classifiers under extreme class skewness and data starvation, we construct a pathological training subset of 1,450 samples. Digit ‘8’ is designated as the majority class with 1,000 samples, while the remaining nine classes are restricted to 50 samples each (a 20:1 imbalance

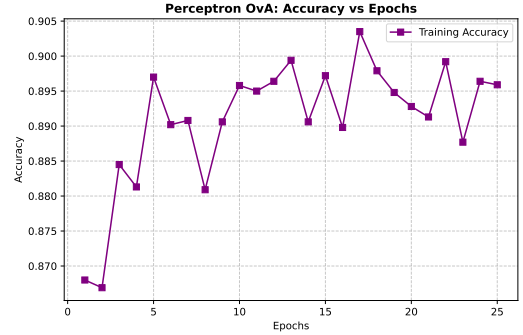


Figure 14. Epoch-wise training accuracy of the OvA Perceptron, demonstrating the necessity for early stopping.

ratio). Class ‘8’ was chosen due to its high geometric overlap and misclassification rate in baseline evaluations, allowing us to examine how decision hyperplanes degrade when biased toward an entangled digit.

Pipeline Integrity: We maintain the optimal *Centered_PCA* pipeline established in Scenario A. Spatial centering is applied independently per image across all partitions, including the hold-out test set. To strictly prevent data leakage, PCA (retaining 95% variance) is fit exclusively on the training folds within the cross-validation loops rather than globally.

Hyperparameter Invalidation: Transitioning from 10,000 balanced samples to 1,450 highly skewed samples fundamentally alters the optimization landscape. Consequently, hyperparameters optimized in Scenario A cannot be transferred and require an independent search.

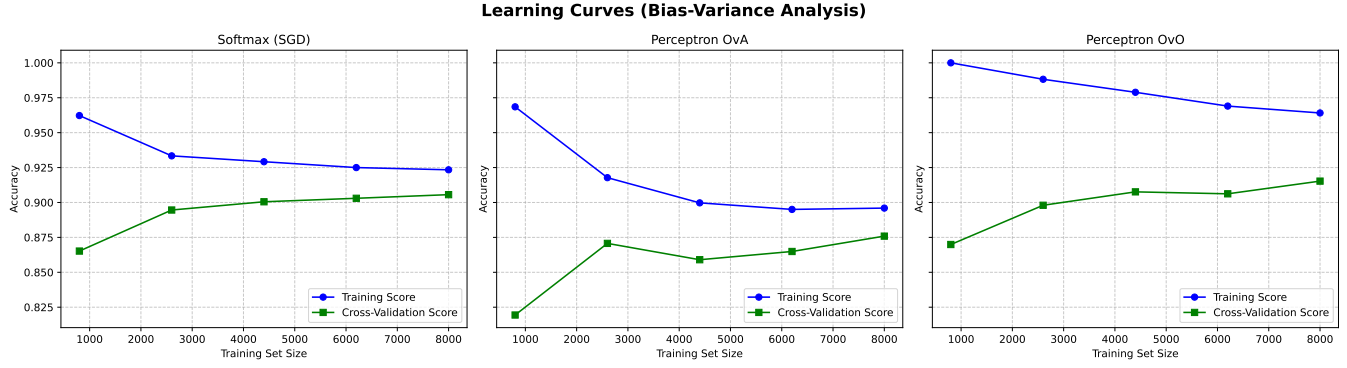


Figure 15. Learning curves illustrating the bias-variance tradeoff for the linear architectures evaluated via 5-fold cross-validation.

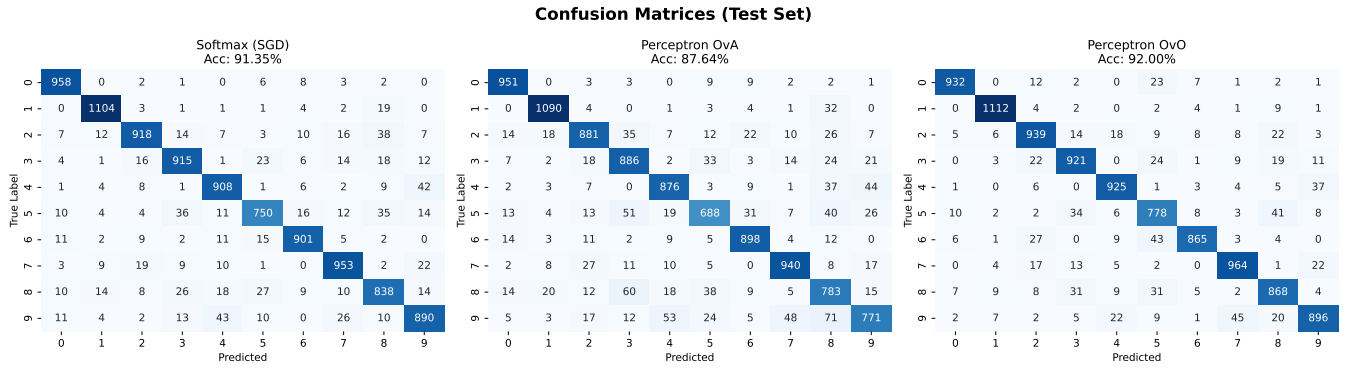


Figure 16. Test Set confusion matrices for the optimized linear classifiers under the balanced Scenario A.

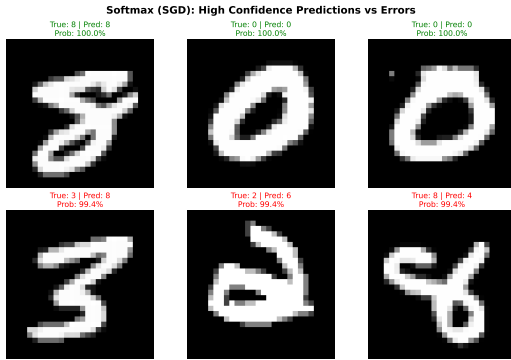


Figure 17. High-confidence predictions versus high-confidence misclassifications for the Softmax (SGD) classifier.

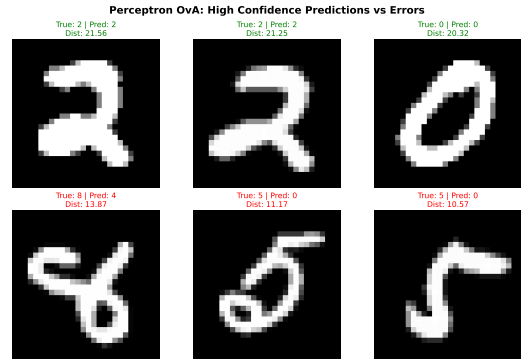


Figure 18. High-confidence predictions versus high-confidence misclassifications for the Perceptron OvA classifier.

9.2. Phase 1: Baseline Evaluation & The Impact of Imbalance

We evaluate the linear models, without applying any class balancing techniques, using Nested Stratified Cross-Validation (5 outer folds, 3 inner folds). The inner loop conducts hyperparameter tuning, while the outer loop provides an unbiased generalization estimate.

Table 4 and Figure 20 expose three key observations:

- **The Accuracy Paradox:** While overall accuracy

remains superficially high (86.69%–91.72%), it is heavily distorted because digit ‘8’ constitutes nearly 69% of the dataset. The Macro F1 scores reveal widespread failure on minority classes, where Softmax reaches an inflated majority recall of 0.9780 by sacrificing minority detection.

- **Architectural Vulnerability (OvA vs. OvO):** Performance degradation depends heavily on ensemble architecture. Perceptron OvA suffers severe boundary collapse: the 20:1 imbalance biases each One-vs-

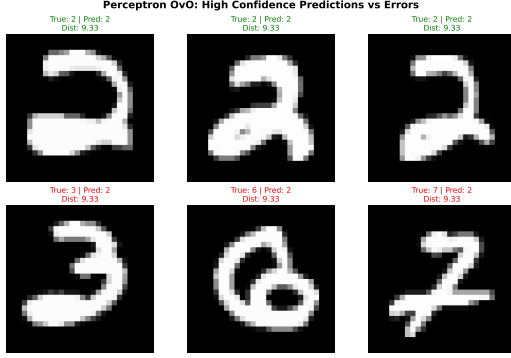


Figure 19. High-confidence predictions versus high-confidence misclassifications for the Perceptron OvO classifier.

Rest classifier toward the dominant negative (“Rest”) class, causing predictions for digit ‘8’ to absorb minority samples (e.g., digit ‘3’ retains only 28 correct classifications). Conversely, Perceptron OvO remains structurally resilient because its binary estimators train exclusively on balanced class pairs, preserving minority boundaries far more effectively (yielding 37 correct classifications for digit ‘3’).

- **Softmax Overconfidence:** Diagnostic analysis reveals a severe calibration issue in the Softmax model. The classifier exhibits 93.99% confidence when predicting the majority class ‘8’, but confidence drops to 73.57% for minority digits. Driven by the multinomial log-loss function, Softmax becomes overly confident in the majority class to minimize aggregate error, effectively shifting its boundaries to “absorb” visually similar minority samples. (Note: Confidence is only evaluated for Softmax, as Perceptrons are pure margin classifiers that do not output native probabilities).

TABLE 4. NESTED CROSS-VALIDATION METRICS ILLUSTRATING THE PERFORMANCE DEGRADATION OF LINEAR ARCHITECTURES UNDER SEVERE CLASS IMBALANCE.

Model	Accuracy	Macro F1
Softmax	0.9028	0.8063
Perceptron OvA	0.8572	0.7090
Perceptron OvO	0.8938	0.7848

9.3. Phase 2: Cost-Sensitive Learning & Test Set Evaluation

To counter the extreme class imbalance, we apply cost-sensitive learning (`class_weight='balanced'`), which penalizes minority misclassifications inversely proportional to class frequencies. Hyperparameters are tuned via Nested CV on the skewed subset, and final performance is evaluated on the balanced 10,000-sample test set.

The results (Table 5 and Figure 21) highlight both the effectiveness and the limitations of cost-sensitive optimization:

- **OvA Recovery:** Weighting prevents Perceptron OvA from defaulting to the majority class, successfully restoring minority decision boundaries and improving overall generalization.
- **Precision-Recall Trade-off:** Pushing boundaries to protect minority classes forces them into the majority class (‘8’) territory. Consequently, recall for ‘8’ remains high (0.96), but precision drops severely (0.57) due to increased false positives.
- **Limits of Data Starvation:** While cost weighting fixes geometric boundary bias, test set improvements remain modest (Macro F1 reaches 0.8266 for OvO). Mathematical penalties cannot synthesize the missing visual variance caused by training on only 50 minority samples.

9.4. The Validation Illusion & Distribution Shift

The Metric Paradox: During Nested CV, the cost-sensitive OvA model yielded a lower Macro F1-score (0.6642) than its naive counterpart (0.7294). However, on the balanced test set, the cost-sensitive model decisively outperformed the naive baseline (0.7907 vs 0.7090).

Mechanics of Validation Bias: Because the cross-validation folds retain the 20:1 imbalance, a model that actively predicts minority classes generates numerous false positives against the abundant majority class, crushing its precision. Consequently, skewed validation sets mathematically reward naive models that simply default to the majority class, producing deceptive metrics.

Empirical Proof: To isolate this effect, we extracted a perfectly balanced validation set (15 samples per class), leaving only 35 samples per minority class for training. Evaluated on this balanced space, the performance gap strictly reversed: the cost-sensitive model achieved a Macro F1 of 0.7443, decisively beating the naive model’s 0.6730.

This empirically proves a core methodological rule: due to distribution shift, validation metrics are fundamentally invalid for model selection unless the validation distribution strictly matches the target real-world distribution.

10. Insights and Conclusions

The evaluation of linear classifiers across both balanced and highly skewed distributions yielded four key methodological insights:

- **Computational Bottlenecks:** While the OvO ensemble simplifies decision boundaries and achieves the highest accuracy, its overall training time is strictly bottlenecked by difficult, highly entangled digit pairs (e.g., 4 vs 9).
- **Implicit Regularization:** Applying PCA to spatially centered images filters structural noise and acts as a

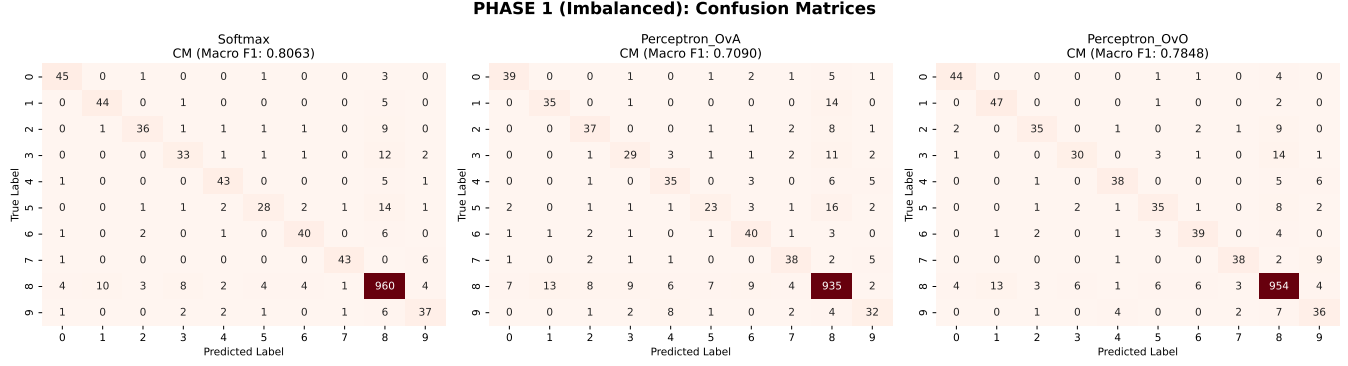


Figure 20. Nested cross-validation confusion matrices illustrating the structural collapse of linear models under extreme class imbalance (Phase 1).

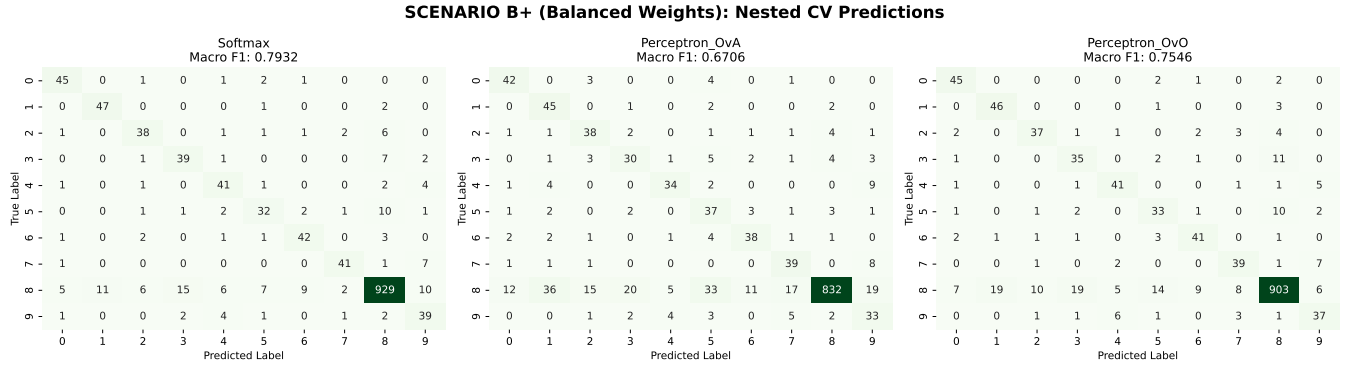


Figure 21. Test set confusion matrices after applying cost-sensitive class weights (Phase 2).

TABLE 5. FINAL EVALUATION METRICS COMPARING VALIDATION (NESTED CV) AND REAL-WORLD (TEST SET) PERFORMANCE AFTER THE APPLICATION OF COST-SENSITIVE CLASS WEIGHTS.

Model	CV F1-score	Test Set F1-score	Test Set Accuracy	Best Params
Softmax	0.7932	0.8555	0.8562	{‘alpha’: 0.01, ‘eta0’: 0.01, ‘penalty’: ‘l2’, ‘learning_rate’: ‘constant’}
Perceptron OvA	0.6706	0.8056	0.8054	{‘eta0’: 0.01, ‘penalty’: None}
Perceptron OvO	0.7546	0.8532	0.8540	{‘alpha’: 0.0001, ‘eta0’: 0.1, ‘penalty’: ‘l2’}

powerful regularizer. This drastically reduces memory footprint and training time without sacrificing accuracy.

- **The Deception of Accuracy:** Scenario B demonstrated that global accuracy is meaningless under extreme class imbalance. Models achieved over 90% accuracy while entirely failing to detect minority classes, confirming Macro F1 as the necessary evaluation metric.
- **The Validation Illusion (Distribution Shift):** Skewed cross-validation folds mathematically reward naive models and penalize cost-sensitive minority predictions. This proves a core rule: validation metrics are fundamentally invalid for model selection if a distribution shift exists between the evaluation space and the target real-world distribution.

Ultimately, while linear parametric models are highly

resource-efficient, their strict reliance on static pixel overlaps prevents them from capturing spatial transformations like rotation or stroke thickness. Achieving near-perfect digit recognition requires transitioning to non-linear architectures.

Declarations

During the preparation of this manuscript, Large Language Models (LLMs) were utilized to assist with code generation, debugging, and language refinement. The originality of the experimental analysis remain entirely intact.

References

- [1] Y. LeCun, C. Cortes, and C. J. Burges, “Mnist handwritten digit database,” *ATT Labs [Online]*. Available: <http://yann.lecun.com/exdb/mnist>, vol. 2, 2010.